Allocate, Load, Install, Launch, and Analyze

High Performance Computing Hands-On Tutorial

Ali Lotfi
Agronomic Crop Imaging Lab

April 1, 2026

Contents

1 Submitting Jobs with SLURM **4**
1.1 The difference between running and submitting . . . 5
1.2 Writing a SLURM job script . . . 6
1.3 Monitoring and cancelling jobs . . . 7
1.4 What happens when the time limit is exceeded . . . 8
1.5 Running a single command on a compute node — `srun` . . . 9
1.6 Checking job efficiency with seff and sacct . . . 9

2 Loading Software with Modules **10**
2.1 Exploring available software . . . 10
2.2 Loading and unloading . . . 10

3 Running Python on the Cluster **11**
3.1 Step 1 — Create the environment . . . 11
3.2 Step 2 — Activate and install packages . . . 12
3.3 Step 3 — Write a Python script . . . 13
3.4 Step 4 — Write the job script . . . 13
3.5 Step 5 — Submit and check output . . . 14

4 Transferring Files with scp **14**
4.1 Uploading to the cluster . . . 14
4.2 Downloading from the cluster . . . 15
4.3 Windows line endings — dos2unix . . . 15
4.4 Large transfers — bundle first with tar . . . 16

5 Running GPU Jobs **17**
5.1 Step 1 — Find available GPUs . . . 17

5.2 Step 2 — CPU vs GPU accounts 19
5.3 Step 3 — Install PyTorch and write the GPU job script 19
5.4 Step 4 — Confirm the GPU is being used 22
5.5 Real result — CIFAR-10 on Fir 22

How to Use These Notes

These notes cover SLURM job submission, software modules, Python virtual environments, file transfer, and GPU jobs on the Fir cluster. Work through the sections in order — each builds on the previous one. Every command was tested live on Fir.

Box guide:

- **Orange "Try it yourself" boxes** — commands to run right now on the cluster
- **Red warning boxes** — common mistakes that cause silent or confusing errors
- **Green tip boxes** — best practices and time-savers
- **Blue note boxes** — extra context that is useful but not critical
- **Yellow intuition boxes** — the mental model behind a concept

Before You Begin

Substitute these placeholders everywhere before typing

```
yourUsername                -> your Alliance username (find
   with: whoami)
yourWindowsUsername         -> your Windows login name
/scratch/yourUsername       -> /scratch/YOUR_ACTUAL_USERNAME
def-kanj241_cpu             -> def-yourPI_cpu
def-kanj241_gpu             -> def-yourPI_gpu
```

Example: if your Alliance username is `jsmith`, then `/scratch/yourUsername/` becomes `/scratch/jsmith/`.

Already logged in?

These notes assume you are already connected to the Fir cluster via SSH. If not, open Command Prompt on Windows and run:

```
ssh yourUsername@fir.alliancecan.ca
```

Authenticate with your Alliance password and a Duo Push (press 1, then approve on your phone).

Create your working directory

All exercises use a single working directory on scratch. Create it now before starting:

Try it yourself

```
$ cd ~/scratch
$ mkdir hpc-tutorial
$ cd hpc-tutorial
$ pwd
/home/yourUsername/scratch/hpc-tutorial
```

Use this directory for everything in these notes. Scratch is fast storage ideal for active work — just remember it is purged after 60 days, so move important results to `projects/` when you are done.

`pwd` shows a different path than the scripts use

You may notice `pwd` prints `/home/yourUsername/scratch/hpc-tutorial`, while the job scripts in this tutorial use `/scratch/yourUsername/hpc-tutorial`. Both paths point to **exactly the same location**.
On DRAC clusters, `~/scratch` is a **symbolic link** (a shortcut) that points to the real scratch filesystem at `/scratch/yourUsername`. When `pwd` follows that symlink it shows the real underlying path. You can confirm this yourself:

```
$ ls -la ~/ | grep scratch
scratch -> /scratch/yourUsername
```

The scripts in this tutorial use `/scratch/yourUsername/` (the real path) because SLURM does not always resolve symlinks reliably. Either form works in the terminal — just be consistent within a single script.

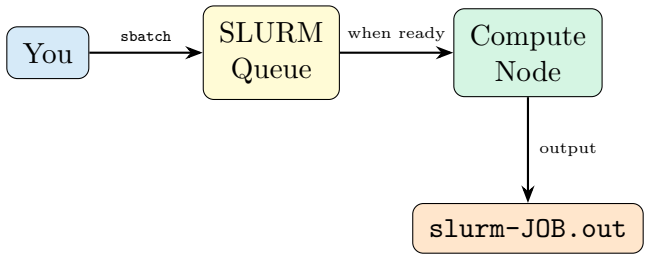
1 Submitting Jobs with SLURM

Goal of this Section

Submit your first job to a compute node, monitor it, and understand what happens when resources run out.

Intuition

The cluster is shared by thousands of users. Without a scheduler, everyone would fight over the same machines. SLURM is the "restaurant manager" — you tell it what you need (CPUs, memory, time), it puts you in a queue, and assigns you a table (compute node) when one becomes available.



1.1 The difference between running and submitting

First, make sure you are in your working directory, then create a simple job script to test with:

```
$ cd /scratch/yourUsername/hpc-tutorial
$ nano example-job.sh
```

```
#!/bin/bash
#SBATCH -J my_test_job
#SBATCH --time=00:05:00
#SBATCH --mem=256M
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

echo -n "This script is running on "
hostname
echo "Job done!"
```

Save with `Ctrl+O`, press Enter, then exit with `Ctrl+X`.

Try it yourself

Now run it *directly* and note where it runs:

```
$ sh example-job.sh
This script is running on login3    # <-- WRONG: login node!
```

Now submit it properly through SLURM:

```
$ sbatch example-job.sh
Submitted batch job 30498427
```

Check the output after it finishes:

```
$ cat slurm-30498427.out
This script is running on fc30519   # <-- CORRECT: compute
```

```
   node!
Job done!
```

1.2 Writing a SLURM job script

A SLURM job script is a shell script with `#SBATCH` directives at the top:

```
#!/bin/bash
#SBATCH -J my_test_job
#SBATCH --time=00:05:00
#SBATCH --mem=1G
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

echo -n "This script is running on "
hostname
echo "Submitted from: $SLURM_SUBMIT_DIR"
echo "Job done!"
```

$SLURM_SUBMIT_DIR

This is an environment variable automatically set by SLURM to the directory you ran `sbatch` from. It is useful for confirming that your job is finding files in the right place.

Key #SBATCH directives:

Directive	Meaning
`-J name`	Job name shown in the queue
`--time=HH:MM:SS`	Maximum walltime. Job is killed if exceeded.
`--mem=XG`	Memory requested (e.g. `1G`, `8G`)
`--ntasks=1`	Number of parallel tasks
`--account=def-kanj241_cpu`	Your PI's CPU allocation account
`--cpus-per-task=N`	CPU cores per task

$SBATCH vs #SBATCH

```
$SBATCH --time=00:05:00     # WRONG: treated as empty
   variable, ignored!
#SBATCH --time=00:05:00     # CORRECT: read by SLURM as a
   directive
```

Using `$` instead of `#` is a silent mistake — SLURM ignores the line and uses defaults.

⚠ No blank lines between `#SBATCH` directives

SLURM stops reading directives as soon as it hits a blank line. Any `#SBATCH` line after a blank line is silently ignored:

```
#SBATCH --time=00:05:00
                            <- blank line: SLURM stops reading
                               here!
#SBATCH --mem=1G            <- silently ignored
#SBATCH --account=def-kanj241_cpu  <- silently ignored
```

This is a common problem when editing scripts in Windows Notepad, which adds blank lines on paste. Keep all `#SBATCH` directives together at the top with no blank lines between them.

1.3 Monitoring and cancelling jobs

```
$ sbatch example-job.sh          # submit
Submitted batch job 30498427

$ squeue -u yourUsername                # check your jobs
JOBID      NAME         ST  TIME_LEFT  NODELIST
30498427   my_test_job  PD  5:00       (Priority)

$ scancel 30498427               # cancel a specific job
$ scancel -u yourUsername               # cancel ALL your jobs
```

Job states:

State	Code	Meaning
Pending	`PD`	Waiting in queue for a node
Running	`R`	Executing on a compute node
Completed	—	Finished; disappears from queue
Timeout	—	Killed by SLURM at time limit
Cancelled	—	Manually cancelled with `scancel`

ℹ Queue reasons

- `(Priority)` — other jobs have higher priority; you are waiting in line
- `(Resources)` — not enough free nodes right now
- `(None)` — just entered the queue

No output file for cancelled pending jobs

If you cancel a job while it is still `PD`, no output file is created — the job never ran on a compute node, so there is nothing to write.

1.4 What happens when the time limit is exceeded

Try it yourself

Create a job script that requests 1 minute but sleeps for 2 minutes:

```
$ nano timeout-test.sh
```

```
#!/bin/bash
#SBATCH -J timeout_test
#SBATCH --time=00:01:00
#SBATCH --mem=256M
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

echo -n "This script is running on "
hostname
sleep 120
echo "This script finished successfully."
```

```
$ sbatch timeout-test.sh
```

After 1 minute, check the output:

```
$ cat slurm-30503950.out
This script is running on fc30613
slurmstepd: error: *** JOB 30503950 ON fc30613
CANCELLED AT 2026-03-29T12:48:16 DUE TO TIME LIMIT ***
```

Notice that **"This script finished successfully." never printed** — SLURM killed the job before it got there. This is why the last `echo` in every job script acts as a **success indicator**: if you see it, the job completed.

Time limit strategy

Estimate how long your job takes, then add a 50% buffer. If unsure, request more time — unused time costs you nothing. For long jobs, save checkpoints periodically in your Python script so progress is not lost if the job times out.

```
#SBATCH --time=00:30:00     # 30 minutes
#SBATCH --time=02:00:00     # 2 hours
#SBATCH --time=1-00:00:00   # 1 day
```

1.5 Running a single command on a compute node — srun

For a quick one-off command that needs a compute node but doesn't need a full job script:

```
$ srun hostname            # run hostname on a compute node
fc30519

$ srun -n 2 echo "hello"   # run on 2 CPUs
hello
hello
```

Note

`srun` blocks your terminal until the command finishes — unlike `sbatch` which returns immediately. Use it for quick tests only, not long computations.

1.6 Checking job efficiency with seff and sacct

```
$ seff 30508122
Job ID: 30508122
State: COMPLETED (exit code 0)
Job Wall-clock time: 00:00:02
Memory Utilized: 19.67 MB
Memory Efficiency: 0.96% of 2.00 GB
```

This job used only 19.67 MB of the 2 GB requested. For the next run, request `--mem=100M` instead.

```
$ sacct               # show all recent jobs
$ sacct -j 30508122   # details for one specific job
```

Tuning workflow

1. First run: request generously (e.g. `--mem=8G`, `--time=02:00:00`)
2. Run `seff JOBID` after it finishes
3. Set future requests to actual usage + 20–30% buffer

2 Loading Software with Modules

Goal of this Section

Load the correct version of Python (and other software) on the cluster before running jobs.

Intuition

Thousands of researchers use the cluster with different software needs. If everyone's software was loaded at once, there would be version conflicts. The **module system** lets each user load exactly the software they need, at the version they need.
Think of it like a library: the books (software) are all on the shelves, but you have to check them out (`module load`) before you can use them.

Software is not loaded by default

When you log in, almost no software is pre-loaded. Even Python may not be the version you expect. Always explicitly load what you need — especially inside job scripts.

2.1 Exploring available software

```
$ module avail              # list all available software
$ module avail python       # search for python specifically
$ module spider python      # find all versions with details
$ module list               # show what is currently loaded
```

2.2 Loading and unloading

```
$ module load python/3.11.5    # load a specific version
$ python --version             # confirm
Python 3.11.5

$ module unload python         # unload
$ module purge                 # unload all non-core modules
$ which python                 # confirm which python is active
```

Always use a specific version

```
module load python/3.11.5   # specific - reproducible
module load python           # default - may change over time
```

If the cluster is updated and the default Python version changes, scripts using just `module load python` may break silently. Pinning to a version keeps your results reproducible.

Load modules inside your job script

What you load on the login node is **not** available on compute nodes. Every job script must load its own modules:

```
#!/bin/bash
#SBATCH ...
module load python/3.11.5    # REQUIRED in every job script
python my_script.py
```

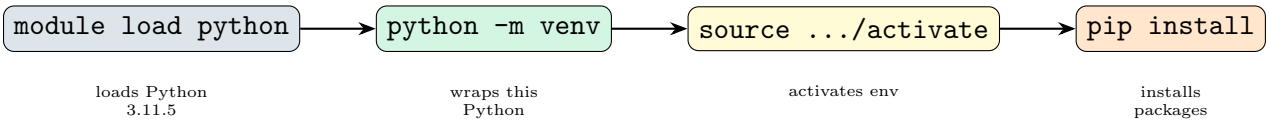
3 Running Python on the Cluster

Goal of this Section

Set up a Python virtual environment, install packages, write a script, and run it on a compute node.

Intuition

A **virtual environment** is an isolated Python installation for your project. It lets you install packages (like numpy or PyTorch) without affecting other users or other projects. Think of it as a dedicated toolbox for one project.
On the cluster, `venv` is the preferred tool (not conda). Unlike conda — which downloads a fresh Python — `venv` *wraps* the Python that is currently loaded. This is why you must `module load python` *before* creating the environment.



3.1 Step 1 — Create the environment

Try it yourself

```
$ module load python/3.11.5
$ python -m venv /scratch/yourUsername/my_env
```

Where to put your environment

```
# Good - permanent, backed up
$ python -m venv
   /home/yourUsername/projects/def-yourPI/envs/my_env

# OK for testing only - purged after 60 days!
$ python -m venv /scratch/yourUsername/my_env
```

3.2 Step 2 — Activate and install packages

Try it yourself

```
$ source /scratch/yourUsername/my_env/bin/activate
(my_env) $ pip install numpy
Successfully installed numpy-2.4.2+computecanada

(my_env) $ python -c "import numpy; print(numpy.__version__)"
2.4.2

(my_env) $ deactivate
```

When the environment is active, your prompt shows `(my_env)` at the start. After `deactivate`, packages are no longer available — confirmed by:

```
$ python -c "import numpy"
ModuleNotFoundError: No module named 'numpy'   # expected!
```

DRAC's wheel cache

Common packages like numpy, scipy, and PyTorch are pre-built for the cluster in a local cache. Running `pip install numpy` installs instantly from this cache rather than downloading from the internet.

3.3 Step 3 — Write a Python script

Try it yourself

Make sure you are in your working directory first, then create the script:

```
$ cd /scratch/yourUsername/hpc-tutorial
$ nano test_script.py
```

```
import numpy as np

print("Python is working on the cluster!")
arr = np.array([1, 2, 3, 4, 5])
print(f"Array: {arr}")
print(f"Mean: {np.mean(arr)}")
print(f"numpy version: {np.__version__}")
```

3.4 Step 4 — Write the job script

Save the following as `python-job.sh` (use `nano python-job.sh`):

```
#!/bin/bash
#SBATCH -J python_test
#SBATCH --time=00:05:00
#SBATCH --mem=1G
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

module load python/3.11.5
source /scratch/yourUsername/my_env/bin/activate
python /scratch/yourUsername/hpc-tutorial/test_script.py
echo "Job done!"
```

Never use ~ in SLURM scripts

SLURM does not expand `~`. Always use full absolute paths confirmed with `pwd`. The `source` command for your virtual environment must also use the full path.
Because `pwd` inside your working directory shows `/home/yourUsername/scratch/hpc-tutorial`, you might be tempted to use that path in a job script — but the real path (and the one to use) is `/scratch/yourUsername/hpc-tutorial`. See the symlink note in "Before You Begin".

3.5 Step 5 — Submit and check output

Try it yourself

```
$ sbatch python-job.sh
Submitted batch job 30508122

$ squeue -u yourUsername
JOBID      NAME          ST  TIME_LEFT
30508122   python_test   PD  5:00

$ cat slurm-30508122.out
Python is working on the cluster!
Array: [1 2 3 4 5]
Mean: 3.0
numpy version: 2.4.2
Job done!
```

4 Transferring Files with scp

Goal of this Section

Upload files from your Windows laptop to the cluster and download results back to your laptop.

Intuition

`scp` (secure copy) works exactly like copying a file on your laptop — except one end of the copy is on a remote machine. The syntax is always: `scp source destination`, where the remote side uses the format `username@hostname:path`.

What does scp stand for?

Secure **C**opy — uses the same encrypted SSH connection as your terminal session.

Note

Run `scp` from your **local PowerShell or Command Prompt window**, not from the SSH session. `scp` requires Duo authentication just like SSH.

4.1 Uploading to the cluster

```
# Upload a single file
```

```
scp "C:\Users\yourWindowsUsername\script.py"
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/

# Upload a whole folder
scp -r "C:\Users\yourWindowsUsername\my_folder"
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/
```

Why not ~/scratch in scp?

On your local machine, ~ expands to your *local* home directory, not the cluster's. Always use the full remote path (`/scratch/yourUsername/`) in `scp` commands to avoid uploading to the wrong place.

4.2 Downloading from the cluster

```
# Download a single file
scp
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/results.txt
   "C:\Users\yourWindowsUsername\"

# Download a whole folder
scp -r
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/results/
   "C:\Users\yourWindowsUsername\"
```

4.3 Windows line endings — dos2unix

Files created on Windows have a hidden `\r` character at the end of every line. On Linux this shows up as `^M`:

Try it yourself

```
$ cat -A example-job.sh
#!/bin/bash^M$
#SBATCH -J my_test_job^M$
#SBATCH --time=00:05:00^M$      <- ^M means Windows line
   ending

$ dos2unix example-job.sh
dos2unix: converting file example-job.sh to Unix format...

$ cat -A example-job.sh
#!/bin/bash$
```

```
#SBATCH -J my_test_job$
#SBATCH --time=00:05:00$        <- clean!
```

⚠ Windows line endings break scripts

For plain text files, `^M` is harmless. But for Python scripts and SLURM job scripts, it causes errors like `/bin/bash^M: bad interpreter`. Always run `dos2unix` on any script you transfer from Windows. Alternatively, write your scripts in VS Code (which defaults to Unix line endings) or PyCharm (set `File` → `File Properties` → `Line Separators` → `LF`).

4.4 Large transfers — bundle first with tar

Transferring thousands of small files is slow because each file requires a separate network handshake. Bundle them into a single compressed file first.

On Windows (PowerShell): You must `cd` into the folder *containing* the folder you want to bundle — not into the folder itself — before running `tar`:

```
# Navigate to the PARENT of the folder you want to bundle
cd C:\Users\yourWindowsUsername\Downloads

# Bundle the results folder into results.tar.gz
tar -czvf results.tar.gz results

# Then upload the single archive to the cluster
scp results.tar.gz
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/
```

⚠ You must be outside the folder you are bundling

If you are *inside* `results\` when you run `tar`, it will fail with "Couldn't visit directory". Always `cd` to the parent folder first.

On the cluster (to bundle job outputs before downloading):

```
$ cd /scratch/yourUsername/hpc-tutorial
$ tar -czvf results.tar.gz results/
```

To unpack the archive after downloading:

```
# On Windows PowerShell
tar -xzvf results.tar.gz
```

```
# On the cluster
$ tar -xzvf results.tar.gz
```

For large or frequent transfers, use `rsync` instead of `scp`:

```
rsync -rvzP local/folder/
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/
```

Flags: `r` recursive, `v` verbose, `z` compress, `P` show progress and resume if interrupted.

5 Running GPU Jobs

Goal of this Section

Find what GPUs are available on Fir, request one correctly, and train a real deep learning model.

Intuition

GPU jobs require one piece of information you cannot guess: the exact **GRES name** of the GPU. Always look it up fresh with `sinfo` before writing a job script — GPU configurations on the cluster can change. The account names (`def-kanj241_cpu` and `def-kanj241_gpu`) are fixed for your lab and do not need to be looked up.

5.1 Step 1 — Find available GPUs

Try it yourself

List all GPU partitions and what they offer:

```
$ sinfo -o "%P %G" | grep gpu
gpubase_interac  gpu:nvidia_h100_80gb_hbm3_3g.40gb:4,...
gpubase_bynode_b1 gpu:h100:4(S:0)
gpubase_bygpu_b1 gpu:h100:4(S:2-3)
gpubase_bygpu_b1 gpu:nvidia_h100_80gb_hbm3_3g.40gb:4,...
... (many more partitions)
```

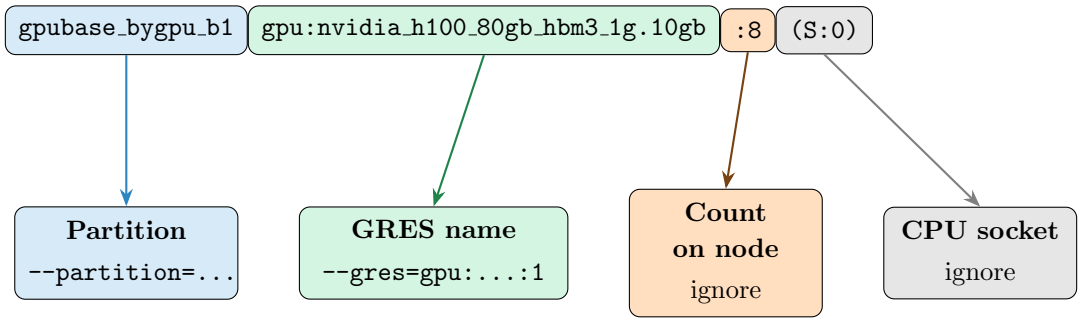
To narrow results to a specific partition, add a second `grep`:

```
$ sinfo -o "%P %G" | grep gpu | grep gpubase_bygpu_b1
gpubase_bygpu_b1 gpu:h100:4(S:2-3)
gpubase_bygpu_b1 gpu:nvidia_h100_80gb_hbm3_3g.40gb:4(S:2-3),...
gpubase_bygpu_b1 gpu:nvidia_h100_80gb_hbm3_3g.40gb:4(S:0),...
```

Why not use the fancier pipeline?

An earlier version of this command used `tr` and `sed` to split each line into one GPU per line. The problem is that `sinfo` only prints the partition name on the *first* GPU in each group — the rest get a blank left side. After splitting, those blank lines no longer match a `grep` filter, so GPU types get silently dropped. The simpler two-`grep` approach above keeps every line intact and filters correctly.

How to read each line:



The number after the GRES name (e.g. `:8` or `:4`) is how many of that GPU type exist on the node — that is the cluster's inventory, not something you set. In your `#SBATCH` line you replace it with how many *you* want, which is almost always `:1`:

```
# sinfo shows:   gpu:nvidia_h100_80gb_hbm3_1g.10gb:8   <- 8
   available on node
# You request:   --gres=gpu:nvidia_h100_80gb_hbm3_1g.10gb:1  <-
   you want 1
```

The `(S:i-j)` suffix refers to which CPU *socket(s)* the GPU is physically connected to on the motherboard. `S:0` means socket 0, `S:2-3` means sockets 2 and 3. This affects memory bandwidth between the CPU and GPU for highly optimised code, but for most jobs you can ignore it entirely — SLURM handles the assignment automatically.

GPU configurations can change

The partitions and GPU types shown by `sinfo` reflect the cluster's current hardware and configuration. DRAC admins can add, remove, or reconfigure them. Always run `sinfo` fresh — never hardcode values from a previous session without verifying.

Fir has NVIDIA H100 80GB GPUs available in four configurations:

GRES name	Memory	Use for
`nvidia_h100_80gb_hbm3_1g.10gb`	10 GB	small models, testing
`nvidia_h100_80gb_hbm3_2g.20gb`	20 GB	medium models
`nvidia_h100_80gb_hbm3_3g.40gb`	40 GB	large models
`h100`	80 GB	very large models

Tip

Always start with the smallest slice that fits your model. Smaller requests are scheduled faster and cost less allocation. A 10 GB slice is more than enough for most image classification tasks.

5.2 Step 2 — CPU vs GPU accounts

On DRAC, CPU and GPU resources use **separate accounts**. You do not need to look these up — they are fixed for your lab:

Account	Use for
`def-kanj241_cpu`	CPU jobs (`sbatch` without `--gres=gpu`)
`def-kanj241_gpu`	GPU jobs (`sbatch` with `--gres=gpu:...`)

Always use the `_cpu` or `_gpu` suffix

Using `def-kanj241` (without `_cpu` or `_gpu`) gives: `Invalid account or account/partition combination`. Always use the specific sub-account.

5.3 Step 3 — Install PyTorch and write the GPU job script

Before writing the job script, install PyTorch into your virtual environment on the login node:

Try it yourself

```
$ module load python/3.11.5
$ source /scratch/yourUsername/my_env/bin/activate
(my_env) $ pip install torch torchvision
Successfully installed torch-2.11.0+computecanada
   torchvision-0.26.0+computecanada
(my_env) $ deactivate
```

Packages install instantly from DRAC's local wheel cache — no internet download needed.

Which CUDA version to load?

Run `module avail cuda` to see what is available on Fir. At the time of writing, `cuda/12.6` is the default. Use `module load cuda` without a version to get the default, or pin to a specific version like `module load cuda/12.6`.

The training script — cifar10_train.py

Make sure you are in your working directory, then create the file:

```
$ cd /scratch/yourUsername/hpc-tutorial
$ nano cifar10_train.py
```

Try it yourself

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# Check GPU availability --- always print this first!
device = torch.device("cuda" if torch.cuda.is_available()
   else "cpu")
print(f"Using device: {device}")

# Download and load CIFAR-10 (downloads automatically on
   first run)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True,
       transform=transform)
trainloader = torch.utils.data.DataLoader(
    trainset, batch_size=64, shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(
    root='./data', train=False, download=True,
       transform=transform)
testloader = torch.utils.data.DataLoader(
    testset, batch_size=64, shuffle=False, num_workers=2)
```

```python
# Define a simple CNN
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool  = nn.MaxPool2d(2, 2)
        self.fc1   = nn.Linear(64 * 8 * 8, 512)
        self.fc2   = nn.Linear(512, 10)
        self.relu  = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 64 * 8 * 8)
        x = self.relu(self.fc1(x))
        return self.fc2(x)

model     = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
epochs = 5
for epoch in range(epochs):
    running_loss = 0.0
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    # Evaluate on test set
    correct, total = 0, 0
    with torch.no_grad():
        for inputs, labels in testloader:
            inputs, labels = inputs.to(device),
               labels.to(device)
            _, predicted = torch.max(model(inputs), 1)
            total   += labels.size(0)
```

```
            correct += (predicted == labels).sum().item()

    print(f"Epoch {epoch+1}/{epochs} | "
          f"Loss: {running_loss/len(trainloader):.3f} | "
          f"Accuracy: {100*correct/total:.1f}%")

print("Training complete!")
torch.save(model.state_dict(), "cifar10_model.pth")
print("Model saved to cifar10_model.pth")
```

Now write the job script (use `nano gpu-job.sh`):

```
#!/bin/bash
#SBATCH -J cifar10_training
#SBATCH --time=00:30:00
#SBATCH --mem=8G
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --gres=gpu:nvidia_h100_80gb_hbm3_1g.10gb:1
#SBATCH --partition=gpubase_bygpu_b1
#SBATCH --account=def-kanj241_gpu

module load python/3.11.5
module load cuda
source /scratch/yourUsername/my_env/bin/activate
python /scratch/yourUsername/hpc-tutorial/cifar10_train.py
echo "Job done!"
```

5.4 Step 4 — Confirm the GPU is being used

Always add this at the top of your Python script:

```
device = torch.device("cuda" if torch.cuda.is_available() else
   "cpu")
print(f"Using device: {device}")
```

If the output shows `Using device: cuda` — the GPU is active. If it shows `cpu` — the module was not loaded or the GRES was not granted.

5.5 Real result — CIFAR-10 on Fir

```
$ cat slurm-30511085.out
Using device: cuda
```

```
Epoch 1/5 | Loss: 1.301 | Accuracy: 64.2%
Epoch 2/5 | Loss: 0.910 | Accuracy: 68.8%
Epoch 3/5 | Loss: 0.720 | Accuracy: 71.5%
Epoch 4/5 | Loss: 0.559 | Accuracy: 71.8%
Epoch 5/5 | Loss: 0.408 | Accuracy: 73.0%
Training complete!
Model saved to cifar10_model.pth
Job done!
```

Loss dropped from 1.301 to 0.408 and accuracy reached 73% in 5 epochs — trained on a real H100 GPU on Fir.